

Proyecto de Grado: IA al servicio de las comunidades

Maestría en Analítica para la Inteligencia de Negocios

Nombres: María Fernanda Izquierdo Aparicio & Ana María Ochoa Muñoz

Fecha: 13 de Mayo de 2025

Este modelo forma parte del proyecto de grado de la Maestría en Analítica para la Inteligencia de Negocios de la Pontificia Universidad Javeriana, enfocado en la aplicación de Inteligencia Artificial al servicio de las comunidades. Su objetivo es clasificar imágenes de plantas en función de su especie usando una arquitectura pre-entrenada RESNET

1. Importe de Librerías

```
In [3]: import json

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

import os
import zipfile
import shutil
import seaborn as sns
import datetime
import cv2

import keras
from keras import models
from keras import layers

import random

import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras import Layer
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.preprocessing import image
```

```

from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing.image import img_to_array, load_img
from tensorflow.keras.callbacks import ReduceLROnPlateau
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.layers import GlobalAveragePooling2D
from tensorflow.keras.utils import to_categorical

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score
from sklearn.metrics import confusion_matrix, accuracy_score, ConfusionMatrixDisplay
from sklearn.metrics import roc_curve, auc
from sklearn.metrics import classification_report

from PIL import Image # Librería para abrir y manipular imágenes

# ignoring warnings
import warnings
warnings.simplefilter("ignore")

```

```

In [4]: #Medir_tiempo_ejecucion():
        inicio = datetime.datetime.now()
        print(f"Inicio: {inicio.strftime('%Y-%m-%d %H:%M:%S.%f')}")

```

Inicio: 2025-05-13 20:30:55.968344

2. Definición de Rutas y Parámetros

```

In [6]: data_dir = 'Datos Finales' # Reemplaza con la ruta real
        metadata_file = 'Metadatos_Finales.csv' # Reemplaza con la ruta real
        image_height = 150
        image_width = 150
        batch_size = 15
        num_classes = None # Se determinará automáticamente del archivo CSV

```

3. Cargar y procesar los metadatos

```

In [8]: try:
        metadata = pd.read_csv(metadata_file, sep=";")
        metadata = metadata.rename(columns={'Nombre del archivo': 'image_id'})

        # Asegúrate de que las columnas 'filename' (o el nombre de tu columna de
        # y 'label' (o el nombre de tu columna de etiquetas) existan.

        if 'image_id' not in metadata.columns or 'Planta_Estado_Español' not in

```

```

        raise ValueError("El archivo CSV debe contener las columnas 'image_id' y 'Planta_Estado_Español'")
    class_names = metadata['Planta_Estado_Español'].unique().tolist()
    num_classes = len(class_names)
    print(f"Se encontraron {num_classes} clases: {class_names}")
except FileNotFoundError:
    print(f"Error: No se encontró el archivo de metadatos en '{metadata_file}'")
    exit()
except ValueError as e:
    print(f"Error al procesar el archivo CSV: {e}")
    exit()

```

Se encontraron 18 clases: ['Yuca_Enferma', 'Yuca_Saludable', 'Fresa_Saludable', 'Papa_Enferma', 'Maíz_Saludable', 'Tomate_Enferma', 'Manzana_Enferma', 'Maíz_Enferma', 'Tomate_Saludable', 'Manzana_Saludable', 'Fresa_Enferma', 'Papa_Saludable', 'Coliflor_Enferma', 'Pepino Cohombro_Saludable', 'Fríjol_Enferma', 'Coliflor_Saludable', 'Fríjol_Saludable', 'Pepino Cohombro_Enferma']

```
In [9]: train, test = train_test_split(metadata, test_size=0.2, random_state=42, shuffle=True)
        train.shape, test.shape
```

```
Out[9]: ((42354, 12), (10589, 12))
```

4. Crear un generador de datos

```
In [11]: datagen = ImageDataGenerator(
        validation_split = 0.2,
        preprocessing_function = None,
        rotation_range = 45,
        zoom_range = 0.2,
        horizontal_flip = True,
        vertical_flip = True,
        fill_mode = 'nearest',
        shear_range = 0.1,
        height_shift_range = 0.1,
        width_shift_range = 0.1
    )

    train_generator = datagen.flow_from_dataframe(
        dataframe=train,
        directory=data_dir,
        x_col='image_id',
        y_col='Planta_Estado_Español',
        target_size=(image_height, image_width),
        batch_size=batch_size,
        class_mode='categorical', # Importante para clasificación multiclase
        subset='training',
        seed=42,
        classes=class_names # Asegurarse del orden de las clases
    )

```

```
validation_generator = datagen.flow_from_dataframe(  
    dataframe=train,  
    directory=data_dir,  
    x_col='image_id',  
    y_col='Planta_Estado_Español',  
    target_size=(image_height, image_width),  
    batch_size=batch_size,  
    class_mode='categorical',  
    subset='validation',  
    seed=42,  
    classes=class_names  
)
```

Found 33880 validated image filenames belonging to 18 classes.

Found 8470 validated image filenames belonging to 18 classes.

5. Cargar el modelo ResNet50 preentrenado (sin la capa superior de clasificación)

```
In [13]: base_model = ResNet50(weights='imagenet', include_top=False, input_shape=(in
```

6. Congelar las capas convolucionales base

```
In [15]: for layer in base_model.layers:  
    layer.trainable = False
```

7. Añadir una nueva capa de clasificación personalizada

```
In [17]: x = base_model.output  
x = GlobalAveragePooling2D()(x)  
# Puedes añadir más capas densas aquí si lo deseas  
# x = Dense(1024, activation='relu')(x)  
predictions = Dense(num_classes, activation='softmax')(x)
```

8. Crear el modelo final

```
In [19]: model = Model(inputs=base_model.input, outputs=predictions)
```

9. Compilar el modelo para fine-tuning de la última

capa

```
In [21]: model.compile(optimizer='adam',
                      loss='categorical_crossentropy',
                      metrics=['accuracy'])
```

```
In [22]: model.summary()
```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 150, 150, 3)	0	–
conv1_pad (ZeroPadding2D)	(None, 156, 156, 3)	0	input_layer[0]
conv1_conv (Conv2D)	(None, 75, 75, 64)	9,472	conv1_pad[0][0]
conv1_bn (BatchNormalizatio...	(None, 75, 75, 64)	256	conv1_conv[0]
conv1_relu (Activation)	(None, 75, 75, 64)	0	conv1_bn[0][0]
pool1_pad (ZeroPadding2D)	(None, 77, 77, 64)	0	conv1_relu[0]
pool1_pool (MaxPooling2D)	(None, 38, 38, 64)	0	pool1_pad[0][0]
conv2_block1_1_conv (Conv2D)	(None, 38, 38, 64)	4,160	pool1_pool[0]
conv2_block1_1_bn (BatchNormalizatio...	(None, 38, 38, 64)	256	conv2_block1_1_conv[0]
conv2_block1_1_relu (Activation)	(None, 38, 38, 64)	0	conv2_block1_1_bn[0]
conv2_block1_2_conv (Conv2D)	(None, 38, 38, 64)	36,928	conv2_block1_1_relu[0]
conv2_block1_2_bn (BatchNormalizatio...	(None, 38, 38, 64)	256	conv2_block1_2_conv[0]
conv2_block1_2_relu (Activation)	(None, 38, 38, 64)	0	conv2_block1_2_bn[0]

conv2_block1_0_conv (Conv2D)	(None, 38, 38, 256)	16,640	pool1_pool[0]
conv2_block1_3_conv (Conv2D)	(None, 38, 38, 256)	16,640	conv2_block1_
conv2_block1_0_bn (BatchNormalizatio...	(None, 38, 38, 256)	1,024	conv2_block1_
conv2_block1_3_bn (BatchNormalizatio...	(None, 38, 38, 256)	1,024	conv2_block1_
conv2_block1_add (Add)	(None, 38, 38, 256)	0	conv2_block1_ conv2_block1_
conv2_block1_out (Activation)	(None, 38, 38, 256)	0	conv2_block1_
conv2_block2_1_conv (Conv2D)	(None, 38, 38, 64)	16,448	conv2_block1_
conv2_block2_1_bn (BatchNormalizatio...	(None, 38, 38, 64)	256	conv2_block2_
conv2_block2_1_relu (Activation)	(None, 38, 38, 64)	0	conv2_block2_
conv2_block2_2_conv (Conv2D)	(None, 38, 38, 64)	36,928	conv2_block2_
conv2_block2_2_bn (BatchNormalizatio...	(None, 38, 38, 64)	256	conv2_block2_
conv2_block2_2_relu (Activation)	(None, 38, 38, 64)	0	conv2_block2_
conv2_block2_3_conv (Conv2D)	(None, 38, 38, 256)	16,640	conv2_block2_
conv2_block2_3_bn (BatchNormalizatio...	(None, 38, 38, 256)	1,024	conv2_block2_
conv2_block2_add (Add)	(None, 38, 38, 256)	0	conv2_block1_ conv2_block2_
conv2_block2_out (Activation)	(None, 38, 38, 256)	0	conv2_block2_
conv2_block3_1_conv (Conv2D)	(None, 38, 38, 64)	16,448	conv2_block2_
conv2_block3_1_bn (BatchNormalizatio...	(None, 38, 38, 64)	256	conv2_block3_

conv2_block3_1_relu (Activation)	(None, 38, 38, 64)	0	conv2_block3_
conv2_block3_2_conv (Conv2D)	(None, 38, 38, 64)	36,928	conv2_block3_
conv2_block3_2_bn (BatchNormalizatio...	(None, 38, 38, 64)	256	conv2_block3_
conv2_block3_2_relu (Activation)	(None, 38, 38, 64)	0	conv2_block3_
conv2_block3_3_conv (Conv2D)	(None, 38, 38, 256)	16,640	conv2_block3_
conv2_block3_3_bn (BatchNormalizatio...	(None, 38, 38, 256)	1,024	conv2_block3_
conv2_block3_add (Add)	(None, 38, 38, 256)	0	conv2_block2_ conv2_block3_
conv2_block3_out (Activation)	(None, 38, 38, 256)	0	conv2_block3_
conv3_block1_1_conv (Conv2D)	(None, 19, 19, 128)	32,896	conv2_block3_
conv3_block1_1_bn (BatchNormalizatio...	(None, 19, 19, 128)	512	conv3_block1_
conv3_block1_1_relu (Activation)	(None, 19, 19, 128)	0	conv3_block1_
conv3_block1_2_conv (Conv2D)	(None, 19, 19, 128)	147,584	conv3_block1_
conv3_block1_2_bn (BatchNormalizatio...	(None, 19, 19, 128)	512	conv3_block1_
conv3_block1_2_relu (Activation)	(None, 19, 19, 128)	0	conv3_block1_
conv3_block1_0_conv (Conv2D)	(None, 19, 19, 512)	131,584	conv2_block3_
conv3_block1_3_conv (Conv2D)	(None, 19, 19, 512)	66,048	conv3_block1_
conv3_block1_0_bn (BatchNormalizatio...	(None, 19, 19, 512)	2,048	conv3_block1_
conv3_block1_3_bn	(None, 19, 19,	2,048	conv3_block1_

(BatchNormalizatio...	512)		
conv3_block1_add (Add)	(None, 19, 19, 512)	0	conv3_block1_ conv3_block1_
conv3_block1_out (Activation)	(None, 19, 19, 512)	0	conv3_block1_
conv3_block2_1_conv (Conv2D)	(None, 19, 19, 128)	65,664	conv3_block1_
conv3_block2_1_bn (BatchNormalizatio...	(None, 19, 19, 128)	512	conv3_block2_
conv3_block2_1_relu (Activation)	(None, 19, 19, 128)	0	conv3_block2_
conv3_block2_2_conv (Conv2D)	(None, 19, 19, 128)	147,584	conv3_block2_
conv3_block2_2_bn (BatchNormalizatio...	(None, 19, 19, 128)	512	conv3_block2_
conv3_block2_2_relu (Activation)	(None, 19, 19, 128)	0	conv3_block2_
conv3_block2_3_conv (Conv2D)	(None, 19, 19, 512)	66,048	conv3_block2_
conv3_block2_3_bn (BatchNormalizatio...	(None, 19, 19, 512)	2,048	conv3_block2_
conv3_block2_add (Add)	(None, 19, 19, 512)	0	conv3_block1_ conv3_block2_
conv3_block2_out (Activation)	(None, 19, 19, 512)	0	conv3_block2_
conv3_block3_1_conv (Conv2D)	(None, 19, 19, 128)	65,664	conv3_block2_
conv3_block3_1_bn (BatchNormalizatio...	(None, 19, 19, 128)	512	conv3_block3_
conv3_block3_1_relu (Activation)	(None, 19, 19, 128)	0	conv3_block3_
conv3_block3_2_conv (Conv2D)	(None, 19, 19, 128)	147,584	conv3_block3_
conv3_block3_2_bn (BatchNormalizatio...	(None, 19, 19, 128)	512	conv3_block3_

conv3_block3_2_relu (Activation)	(None, 19, 19, 128)	0	conv3_block3_
conv3_block3_3_conv (Conv2D)	(None, 19, 19, 512)	66,048	conv3_block3_
conv3_block3_3_bn (BatchNormalizatio...	(None, 19, 19, 512)	2,048	conv3_block3_
conv3_block3_add (Add)	(None, 19, 19, 512)	0	conv3_block2_ conv3_block3_
conv3_block3_out (Activation)	(None, 19, 19, 512)	0	conv3_block3_
conv3_block4_1_conv (Conv2D)	(None, 19, 19, 128)	65,664	conv3_block3_
conv3_block4_1_bn (BatchNormalizatio...	(None, 19, 19, 128)	512	conv3_block4_
conv3_block4_1_relu (Activation)	(None, 19, 19, 128)	0	conv3_block4_
conv3_block4_2_conv (Conv2D)	(None, 19, 19, 128)	147,584	conv3_block4_
conv3_block4_2_bn (BatchNormalizatio...	(None, 19, 19, 128)	512	conv3_block4_
conv3_block4_2_relu (Activation)	(None, 19, 19, 128)	0	conv3_block4_
conv3_block4_3_conv (Conv2D)	(None, 19, 19, 512)	66,048	conv3_block4_
conv3_block4_3_bn (BatchNormalizatio...	(None, 19, 19, 512)	2,048	conv3_block4_
conv3_block4_add (Add)	(None, 19, 19, 512)	0	conv3_block3_ conv3_block4_
conv3_block4_out (Activation)	(None, 19, 19, 512)	0	conv3_block4_
conv4_block1_1_conv (Conv2D)	(None, 10, 10, 256)	131,328	conv3_block4_
conv4_block1_1_bn (BatchNormalizatio...	(None, 10, 10, 256)	1,024	conv4_block1_
conv4_block1_1_relu (Activation)	(None, 10, 10, 256)	0	conv4_block1_

conv4_block1_2_conv (Conv2D)	(None, 10, 10, 256)	590,080	conv4_block1_
conv4_block1_2_bn (BatchNormalizatio...	(None, 10, 10, 256)	1,024	conv4_block1_
conv4_block1_2_relu (Activation)	(None, 10, 10, 256)	0	conv4_block1_
conv4_block1_0_conv (Conv2D)	(None, 10, 10, 1024)	525,312	conv3_block4_
conv4_block1_3_conv (Conv2D)	(None, 10, 10, 1024)	263,168	conv4_block1_
conv4_block1_0_bn (BatchNormalizatio...	(None, 10, 10, 1024)	4,096	conv4_block1_
conv4_block1_3_bn (BatchNormalizatio...	(None, 10, 10, 1024)	4,096	conv4_block1_
conv4_block1_add (Add)	(None, 10, 10, 1024)	0	conv4_block1_ conv4_block1_
conv4_block1_out (Activation)	(None, 10, 10, 1024)	0	conv4_block1_
conv4_block2_1_conv (Conv2D)	(None, 10, 10, 256)	262,400	conv4_block1_
conv4_block2_1_bn (BatchNormalizatio...	(None, 10, 10, 256)	1,024	conv4_block2_
conv4_block2_1_relu (Activation)	(None, 10, 10, 256)	0	conv4_block2_
conv4_block2_2_conv (Conv2D)	(None, 10, 10, 256)	590,080	conv4_block2_
conv4_block2_2_bn (BatchNormalizatio...	(None, 10, 10, 256)	1,024	conv4_block2_
conv4_block2_2_relu (Activation)	(None, 10, 10, 256)	0	conv4_block2_
conv4_block2_3_conv (Conv2D)	(None, 10, 10, 1024)	263,168	conv4_block2_
conv4_block2_3_bn (BatchNormalizatio...	(None, 10, 10, 1024)	4,096	conv4_block2_
conv4_block2_add	(None, 10, 10,	0	conv4_block1_

(Add)	1024)		conv4_block2_
conv4_block2_out (Activation)	(None, 10, 10, 1024)	0	conv4_block2_
conv4_block3_1_conv (Conv2D)	(None, 10, 10, 256)	262,400	conv4_block2_
conv4_block3_1_bn (BatchNormalizatio...	(None, 10, 10, 256)	1,024	conv4_block3_
conv4_block3_1_relu (Activation)	(None, 10, 10, 256)	0	conv4_block3_
conv4_block3_2_conv (Conv2D)	(None, 10, 10, 256)	590,080	conv4_block3_
conv4_block3_2_bn (BatchNormalizatio...	(None, 10, 10, 256)	1,024	conv4_block3_
conv4_block3_2_relu (Activation)	(None, 10, 10, 256)	0	conv4_block3_
conv4_block3_3_conv (Conv2D)	(None, 10, 10, 1024)	263,168	conv4_block3_
conv4_block3_3_bn (BatchNormalizatio...	(None, 10, 10, 1024)	4,096	conv4_block3_
conv4_block3_add (Add)	(None, 10, 10, 1024)	0	conv4_block2_ conv4_block3_
conv4_block3_out (Activation)	(None, 10, 10, 1024)	0	conv4_block3_
conv4_block4_1_conv (Conv2D)	(None, 10, 10, 256)	262,400	conv4_block3_
conv4_block4_1_bn (BatchNormalizatio...	(None, 10, 10, 256)	1,024	conv4_block4_
conv4_block4_1_relu (Activation)	(None, 10, 10, 256)	0	conv4_block4_
conv4_block4_2_conv (Conv2D)	(None, 10, 10, 256)	590,080	conv4_block4_
conv4_block4_2_bn (BatchNormalizatio...	(None, 10, 10, 256)	1,024	conv4_block4_
conv4_block4_2_relu (Activation)	(None, 10, 10, 256)	0	conv4_block4_

conv4_block4_3_conv (Conv2D)	(None, 10, 10, 1024)	263,168	conv4_block4_
conv4_block4_3_bn (BatchNormalizatio...	(None, 10, 10, 1024)	4,096	conv4_block4_
conv4_block4_add (Add)	(None, 10, 10, 1024)	0	conv4_block3_ conv4_block4_
conv4_block4_out (Activation)	(None, 10, 10, 1024)	0	conv4_block4_
conv4_block5_1_conv (Conv2D)	(None, 10, 10, 256)	262,400	conv4_block4_
conv4_block5_1_bn (BatchNormalizatio...	(None, 10, 10, 256)	1,024	conv4_block5_
conv4_block5_1_relu (Activation)	(None, 10, 10, 256)	0	conv4_block5_
conv4_block5_2_conv (Conv2D)	(None, 10, 10, 256)	590,080	conv4_block5_
conv4_block5_2_bn (BatchNormalizatio...	(None, 10, 10, 256)	1,024	conv4_block5_
conv4_block5_2_relu (Activation)	(None, 10, 10, 256)	0	conv4_block5_
conv4_block5_3_conv (Conv2D)	(None, 10, 10, 1024)	263,168	conv4_block5_
conv4_block5_3_bn (BatchNormalizatio...	(None, 10, 10, 1024)	4,096	conv4_block5_
conv4_block5_add (Add)	(None, 10, 10, 1024)	0	conv4_block4_ conv4_block5_
conv4_block5_out (Activation)	(None, 10, 10, 1024)	0	conv4_block5_
conv4_block6_1_conv (Conv2D)	(None, 10, 10, 256)	262,400	conv4_block5_
conv4_block6_1_bn (BatchNormalizatio...	(None, 10, 10, 256)	1,024	conv4_block6_
conv4_block6_1_relu (Activation)	(None, 10, 10, 256)	0	conv4_block6_
conv4_block6_2_conv (Conv2D)	(None, 10, 10, 256)	590,080	conv4_block6_

conv4_block6_2_bn (BatchNormalizatio...	(None, 10, 10, 256)	1,024	conv4_block6_
conv4_block6_2_relu (Activation)	(None, 10, 10, 256)	0	conv4_block6_
conv4_block6_3_conv (Conv2D)	(None, 10, 10, 1024)	263,168	conv4_block6_
conv4_block6_3_bn (BatchNormalizatio...	(None, 10, 10, 1024)	4,096	conv4_block6_
conv4_block6_add (Add)	(None, 10, 10, 1024)	0	conv4_block5_ conv4_block6_
conv4_block6_out (Activation)	(None, 10, 10, 1024)	0	conv4_block6_
conv5_block1_1_conv (Conv2D)	(None, 5, 5, 512)	524,800	conv4_block6_
conv5_block1_1_bn (BatchNormalizatio...	(None, 5, 5, 512)	2,048	conv5_block1_
conv5_block1_1_relu (Activation)	(None, 5, 5, 512)	0	conv5_block1_
conv5_block1_2_conv (Conv2D)	(None, 5, 5, 512)	2,359,808	conv5_block1_
conv5_block1_2_bn (BatchNormalizatio...	(None, 5, 5, 512)	2,048	conv5_block1_
conv5_block1_2_relu (Activation)	(None, 5, 5, 512)	0	conv5_block1_
conv5_block1_0_conv (Conv2D)	(None, 5, 5, 2048)	2,099,200	conv4_block6_
conv5_block1_3_conv (Conv2D)	(None, 5, 5, 2048)	1,050,624	conv5_block1_
conv5_block1_0_bn (BatchNormalizatio...	(None, 5, 5, 2048)	8,192	conv5_block1_
conv5_block1_3_bn (BatchNormalizatio...	(None, 5, 5, 2048)	8,192	conv5_block1_
conv5_block1_add (Add)	(None, 5, 5, 2048)	0	conv5_block1_ conv5_block1_
conv5_block1_out	(None, 5, 5,	0	conv5_block1_

(Activation)	2048)		
conv5_block2_1_conv (Conv2D)	(None, 5, 5, 512)	1,049,088	conv5_block1_
conv5_block2_1_bn (BatchNormalizatio...	(None, 5, 5, 512)	2,048	conv5_block2_
conv5_block2_1_relu (Activation)	(None, 5, 5, 512)	0	conv5_block2_
conv5_block2_2_conv (Conv2D)	(None, 5, 5, 512)	2,359,808	conv5_block2_
conv5_block2_2_bn (BatchNormalizatio...	(None, 5, 5, 512)	2,048	conv5_block2_
conv5_block2_2_relu (Activation)	(None, 5, 5, 512)	0	conv5_block2_
conv5_block2_3_conv (Conv2D)	(None, 5, 5, 2048)	1,050,624	conv5_block2_
conv5_block2_3_bn (BatchNormalizatio...	(None, 5, 5, 2048)	8,192	conv5_block2_
conv5_block2_add (Add)	(None, 5, 5, 2048)	0	conv5_block1_ conv5_block2_
conv5_block2_out (Activation)	(None, 5, 5, 2048)	0	conv5_block2_
conv5_block3_1_conv (Conv2D)	(None, 5, 5, 512)	1,049,088	conv5_block2_
conv5_block3_1_bn (BatchNormalizatio...	(None, 5, 5, 512)	2,048	conv5_block3_
conv5_block3_1_relu (Activation)	(None, 5, 5, 512)	0	conv5_block3_
conv5_block3_2_conv (Conv2D)	(None, 5, 5, 512)	2,359,808	conv5_block3_
conv5_block3_2_bn (BatchNormalizatio...	(None, 5, 5, 512)	2,048	conv5_block3_
conv5_block3_2_relu (Activation)	(None, 5, 5, 512)	0	conv5_block3_
conv5_block3_3_conv (Conv2D)	(None, 5, 5, 2048)	1,050,624	conv5_block3_

conv5_block3_3_bn (BatchNormalizatio...	(None, 5, 5, 2048)	8,192	conv5_block3_
conv5_block3_add (Add)	(None, 5, 5, 2048)	0	conv5_block2_ conv5_block3_
conv5_block3_out (Activation)	(None, 5, 5, 2048)	0	conv5_block3_
global_average_poo... (GlobalAveragePool...	(None, 2048)	0	conv5_block3_
dense (Dense)	(None, 18)	36,882	global_averag

Total params: 23,624,594 (90.12 MB)

Trainable params: 36,882 (144.07 KB)

Non-trainable params: 23,587,712 (89.98 MB)

10. Entrenar el modelo con la última capa descongelada

```
In [24]: early_stopping = EarlyStopping(
    monitor='val_loss', # Monitor the validation loss
    patience=100, # Number of epochs with no improvement after which training will be stopped
    restore_best_weights=True # Restore the model weights from the epoch with the best validation loss
)

from tensorflow.keras.callbacks import ReduceLRonPlateau

reduce_lr = ReduceLRonPlateau(
    monitor='val_loss',
    factor=0.2, # Reduce learning rate by a factor of 0.2
    patience=3, # Number of epochs with no improvement after which learning rate will be reduced
    min_lr=1e-5 # Minimum learning rate
)
```

```
In [25]: epochs_fine_tune = 20 # Define el número de épocas para el fine-tuning
```

```
In [26]: print("Comenzando el entrenamiento de la última capa...")
history_fine_tune = model.fit(
    train_generator,
    steps_per_epoch=train_generator.samples // train_generator.batch_size,
    validation_data=validation_generator,
    validation_steps=validation_generator.samples // validation_generator.batch_size,
    epochs=epochs_fine_tune,
    callbacks=[early_stopping, reduce_lr],
)
```

```
    verbose = 1  
)
```

Comenzando el entrenamiento de la última capa...

Epoch 1/20

2258/2258 ————— **744s** 329ms/step - accuracy: 0.8538 - loss: 0.4859 - val_accuracy: 0.8965 - val_loss: 0.2713 - learning_rate: 0.0010

Epoch 2/20

2258/2258 ————— **153s** 68ms/step - accuracy: 0.8667 - loss: 0.265 - val_accuracy: 0.9017 - val_loss: 0.2662 - learning_rate: 0.0010

Epoch 3/20

2258/2258 ————— **780s** 345ms/step - accuracy: 0.9172 - loss: 0.2402 - val_accuracy: 0.9228 - val_loss: 0.2225 - learning_rate: 0.0010

Epoch 4/20

2258/2258 ————— **156s** 69ms/step - accuracy: 0.8000 - loss: 0.1974 - val_accuracy: 0.9235 - val_loss: 0.2256 - learning_rate: 0.0010

Epoch 5/20

2258/2258 ————— **789s** 350ms/step - accuracy: 0.9230 - loss: 0.2301 - val_accuracy: 0.9098 - val_loss: 0.2616 - learning_rate: 0.0010

Epoch 6/20

2258/2258 ————— **157s** 69ms/step - accuracy: 1.0000 - loss: 0.0356 - val_accuracy: 0.9131 - val_loss: 0.2627 - learning_rate: 0.0010

Epoch 7/20

2258/2258 ————— **788s** 349ms/step - accuracy: 0.9342 - loss: 0.1801 - val_accuracy: 0.9326 - val_loss: 0.1802 - learning_rate: 2.0000e-04

Epoch 8/20

2258/2258 ————— **155s** 69ms/step - accuracy: 0.8667 - loss: 0.2861 - val_accuracy: 0.9301 - val_loss: 0.1881 - learning_rate: 2.0000e-04

Epoch 9/20

2258/2258 ————— **798s** 353ms/step - accuracy: 0.9400 - loss: 0.1634 - val_accuracy: 0.9288 - val_loss: 0.1844 - learning_rate: 2.0000e-04

Epoch 10/20

2258/2258 ————— **160s** 71ms/step - accuracy: 0.9333 - loss: 0.1236 - val_accuracy: 0.9287 - val_loss: 0.1928 - learning_rate: 2.0000e-04

Epoch 11/20

2258/2258 ————— **800s** 354ms/step - accuracy: 0.9438 - loss: 0.1517 - val_accuracy: 0.9371 - val_loss: 0.1765 - learning_rate: 4.0000e-05

Epoch 12/20

2258/2258 ————— **156s** 69ms/step - accuracy: 1.0000 - loss: 0.0320 - val_accuracy: 0.9368 - val_loss: 0.1688 - learning_rate: 4.0000e-05

Epoch 13/20

2258/2258 ————— **799s** 354ms/step - accuracy: 0.9431 - loss: 0.1493 - val_accuracy: 0.9351 - val_loss: 0.1750 - learning_rate: 4.0000e-05

Epoch 14/20

2258/2258 ————— **157s** 70ms/step - accuracy: 0.9333 - loss: 0.5156 - val_accuracy: 0.9381 - val_loss: 0.1688 - learning_rate: 4.0000e-05

Epoch 15/20

2258/2258 ————— **797s** 353ms/step - accuracy: 0.9419 - loss: 0.1464 - val_accuracy: 0.9394 - val_loss: 0.1728 - learning_rate: 4.0000e-05

Epoch 16/20

2258/2258 ————— 158s 70ms/step - accuracy: 0.9333 - loss: 0.127 - val_accuracy: 0.9362 - val_loss: 0.1761 - learning_rate: 1.0000e-05
Epoch 17/20

2258/2258 ————— 803s 356ms/step - accuracy: 0.9448 - loss: 0.1432 - val_accuracy: 0.9365 - val_loss: 0.1702 - learning_rate: 1.0000e-05
Epoch 18/20

2258/2258 ————— 157s 69ms/step - accuracy: 0.9333 - loss: 0.1516 - val_accuracy: 0.9336 - val_loss: 0.1705 - learning_rate: 1.0000e-05
Epoch 19/20

2258/2258 ————— 797s 353ms/step - accuracy: 0.9439 - loss: 0.1447 - val_accuracy: 0.9366 - val_loss: 0.1708 - learning_rate: 1.0000e-05
Epoch 20/20

2258/2258 ————— 157s 69ms/step - accuracy: 0.8667 - loss: 0.1923 - val_accuracy: 0.9366 - val_loss: 0.1706 - learning_rate: 1.0000e-05

```
In [27]: fin = datetime.datetime.now()
print(f"Fin: {fin.strftime('%Y-%m-%d %H:%M:%S.%f')}")

duracion = fin - inicio
print(f"Duración: {duracion}")
```

Fin: 2025-05-13 23:08:39.096166
Duración: 2:37:43.127822

11. Opcional: Descongelar algunas de las últimas capas del modelo base para un fine-tuning más profundo

```
In [49]: #unfreeze_layers = 3 # Número de capas a descongelar (contando desde la última)

#for layer in base_model.layers[-unfreeze_layers:]:
#    layer.trainable = True
```

12. Opcional: Recompilar el modelo después de descongelar las capas

```
In [52]: #model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5), # Usar Adam
#                    loss='categorical_crossentropy',
#                    metrics=['accuracy'])
```

13. Opcional: Entrenar el modelo con las capas descongeladas

```
In [59]: #epochs_deep_fine_tune = 10 # Define el número de épocas para el fine-tuning
```

```
#print(f"\nComenzando el fine-tuning profundo de las últimas {unfreeze_layers} capas")
#history_deep_fine_tune = model.fit(
#    train_generator,
#    steps_per_epoch=train_generator.samples // train_generator.batch_size,
#    validation_data=validation_generator,
#    validation_steps=validation_generator.samples // validation_generator.batch_size,
#    epochs=epochs_deep_fine_tune,
#    verbose = 1
#)
```

14. Evaluar el modelo final

```
In [61]: print("\nEvaluando el modelo...")
evaluation_result = model.evaluate(validation_generator)
print(f"Pérdida en la validación: {evaluation_result[0]:.4f}")
print(f"Precisión en la validación: {evaluation_result[1]:.4f}")
```

Evaluando el modelo...

565/565 ————— **149s** 263ms/step – accuracy: 0.9327 – loss: 0.1759

Pérdida en la validación: 0.1726

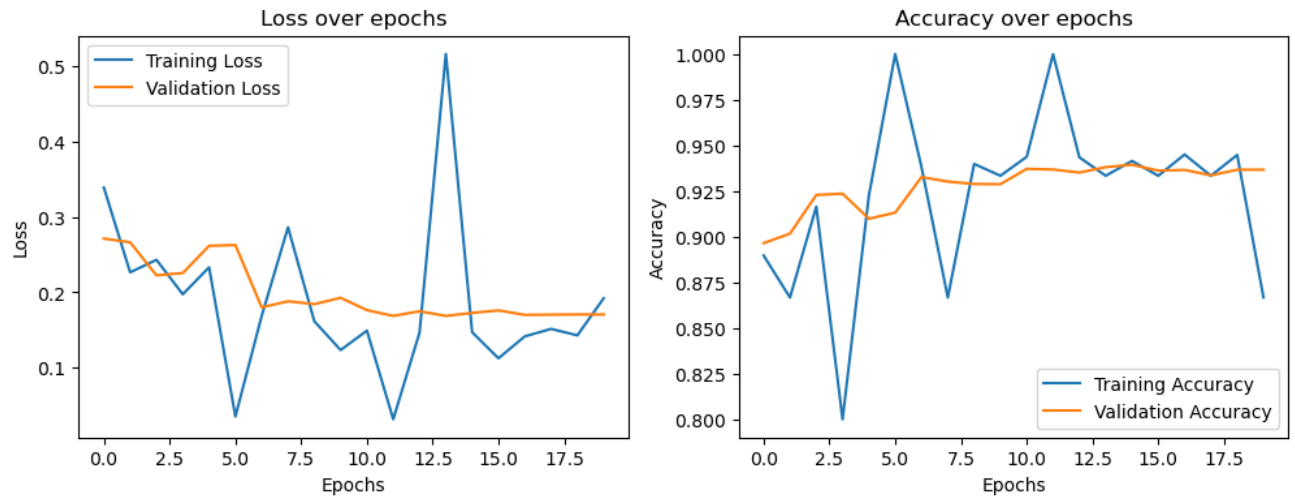
Precisión en la validación: 0.9358

```
In [62]: # Acceder a los datos de pérdida y precisión
loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
accuracy = history_fine_tune.history['accuracy']
val_accuracy = history_fine_tune.history['val_accuracy']

# Graficar la pérdida del modelo
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history_fine_tune.history['loss'], label='Training Loss')
plt.plot(history_fine_tune.history['val_loss'], label='Validation Loss')
plt.title('Loss over epochs')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

# Graficar la precisión del modelo
plt.subplot(1, 2, 2)
plt.plot(history_fine_tune.history['accuracy'], label='Training Accuracy')
plt.plot(history_fine_tune.history['val_accuracy'], label='Validation Accuracy')
plt.title('Accuracy over epochs')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
```

Out[62]: <matplotlib.legend.Legend at 0x284785940>



```
In [63]: epochs = list(range(1, 21))
data = {
    'Epoch': epochs,
    'Training Loss': loss,
    'Validation Loss': val_loss,
    'Training Accuracy': accuracy,
    'Validation Accuracy': val_accuracy
}
df = pd.DataFrame(data)

# Imprimir la tabla
print(df)
```

	Epoch	Trainning Loss	Validation Loss	Training Accuracy \
0	1	0.338819	0.271258	0.889650
1	2	0.226491	0.266232	0.866667
2	3	0.243116	0.222525	0.916403
3	4	0.197422	0.225554	0.800000
4	5	0.233200	0.261617	0.922545
5	6	0.035622	0.262671	1.000000
6	7	0.168811	0.180169	0.938107
7	8	0.286113	0.188143	0.866667
8	9	0.161764	0.184446	0.939820
9	10	0.123609	0.192767	0.933333
10	11	0.149292	0.176477	0.943747
11	12	0.031997	0.168782	1.000000
12	13	0.146663	0.175004	0.943452
13	14	0.515576	0.168812	0.933333
14	15	0.147160	0.172822	0.941444
15	16	0.112727	0.176085	0.933333
16	17	0.141688	0.170178	0.944987
17	18	0.151602	0.170458	0.933333
18	19	0.143068	0.170788	0.944751
19	20	0.192285	0.170647	0.866667

	Validation Accuracy
0	0.896454
1	0.901655
2	0.922813
3	0.923522
4	0.909811
5	0.913121
6	0.932624
7	0.930142
8	0.928842
9	0.928723
10	0.937116
11	0.936761
12	0.935106
13	0.938061
14	0.939362
15	0.936170
16	0.936525
17	0.933570
18	0.936643
19	0.936643

```
In [64]: # Accuracy final en entrenamiento
val_loss = history_fine_tune.history['val_loss'][-1]
```

```
In [65]: print('Pérdida final en Train:', history_fine_tune.history['loss'][-1])
print("Pérdida final en Validation:", history_fine_tune.history['val_loss'][-1])
```

```
print("Accuracy final en Train:", history_fine_tune.history['accuracy'][-1])
print("Accuracy final en Validation:", history_fine_tune.history['val_accuracy'][-1])
```

Perdida final en Train: 0.19228483736515045
 Pérdida final en Validation: 0.1706472635269165
 Accuracy final en Train: 0.8666666746139526
 Accuracy final en Validation: 0.9366430044174194

In []:

15. Testeo del Modelo

In [68]: `test.Planta_Estado_Español = test.Planta_Estado_Español.astype('str')`

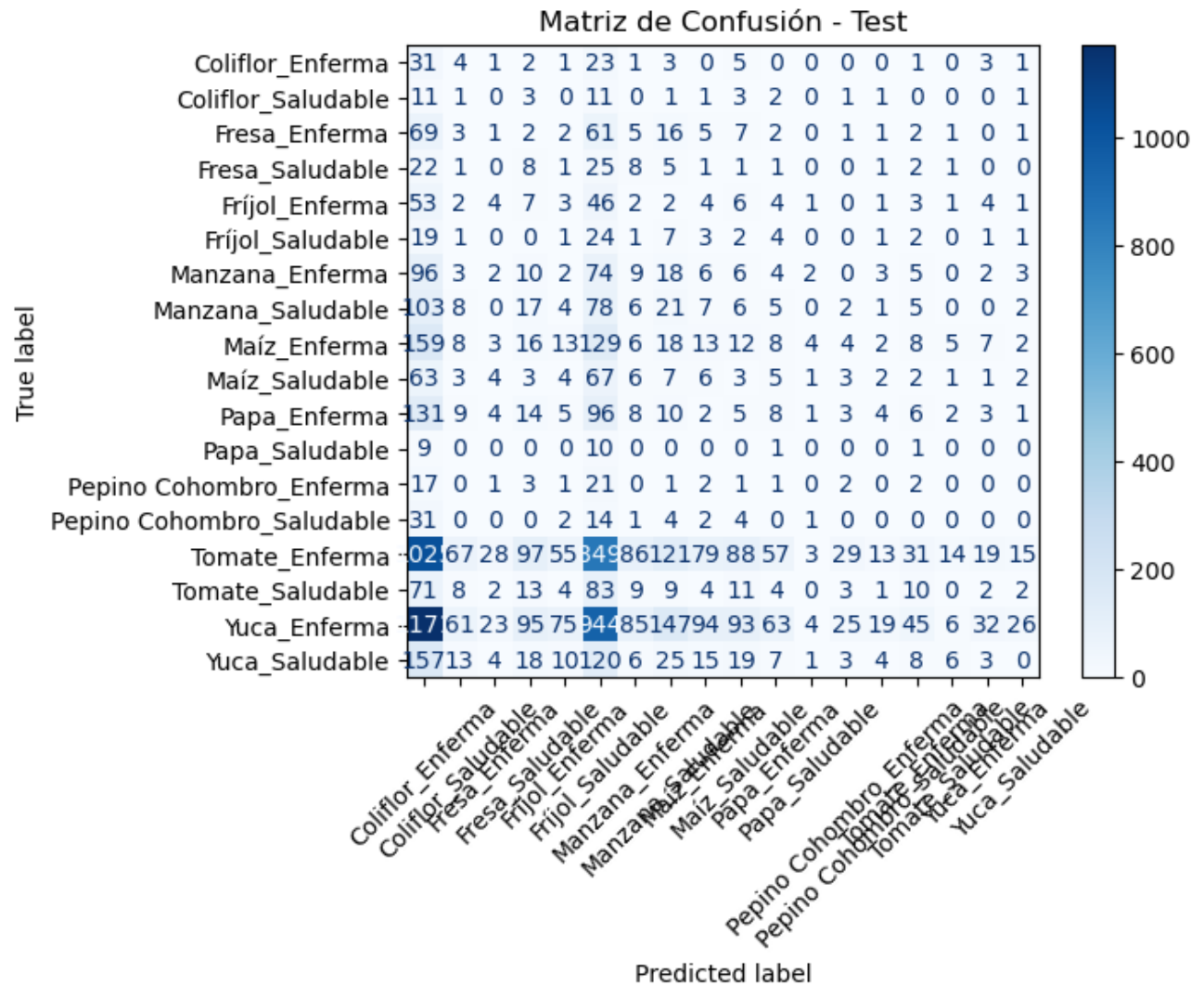
```
test_generator = datagen.flow_from_dataframe(test,
      directory = "Datos Finales",
      subset = "training",
      x_col = "image_id",
      y_col = "Planta_Estado_Español",
      target_size = (image_height, image_width),
      batch_size = batch_size,
      class_mode = "categorical")
```

Found 8472 validated image filenames belonging to 18 classes.

In [69]: `# -----`
1. Obtener predicciones completas para test
`# -----`
`y_pred_probs = model.predict(test_generator)`
`y_pred = np.argmax(y_pred_probs, axis=1)`
`y_true = test_generator.classes`

565/565  151s 265ms/step

In [70]: `# -----`
2. Matriz de Confusión
`# -----`
`cm = confusion_matrix(y_true, y_pred)`
`disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=test_generator.classes)`
`disp.plot(cmap="Blues", xticks_rotation=45)`
`plt.title("Matriz de Confusión - Test")`
`plt.show()`



```
In [71]: accuracy = accuracy_score(y_true, y_pred)
print(f"Test Accuracy: {accuracy:.4f}")
```

Test Accuracy: 0.0221

```
In [72]: # Función para normalizar imágenes solo para visualización
def normalize_for_display(img):
    img_min = img.min()
    img_max = img.max()
    if img_max > 1.0 or img_min < 0.0:
        img = (img - img_min) / (img_max - img_min + 1e-7)
    return img

# Obtener nombres de clases
class_indices = test_generator.class_indices
class_names = list(class_indices.keys())

# Reiniciar el generador para empezar desde el principio
test_generator.reset()
```

```
x_batch, y_true_batch = next(test_generator)

# Obtener predicciones del modelo
y_pred_probs_batch = model.predict(x_batch)
y_pred_batch = np.argmax(y_pred_probs_batch, axis=1)
y_true_batch = np.argmax(y_true_batch, axis=1)

# Número de imágenes a mostrar (máximo el tamaño del batch)
num_samples = x_batch.shape[0]

# Mostrar imágenes con etiquetas
plt.figure(figsize=(15, 8))
for i in range(num_samples):
    plt.subplot(3, 4, i + 1)
    img_to_show = normalize_for_display(x_batch[i])
    plt.imshow(img_to_show)
    plt.axis('off')
    plt.title(
        f"Real: {class_names[y_true_batch[i]]}\nPred: {class_names[y_pred_batch[i]]} "
        f"color='green' if y_true_batch[i] == y_pred_batch[i] else 'red' "
    )
plt.tight_layout()
plt.suptitle("Muestreo de Predicciones vs. Etiquetas Reales", fontsize=16, y=1.05)
plt.show()
```

1/1 ————— 1s 747ms/step

```

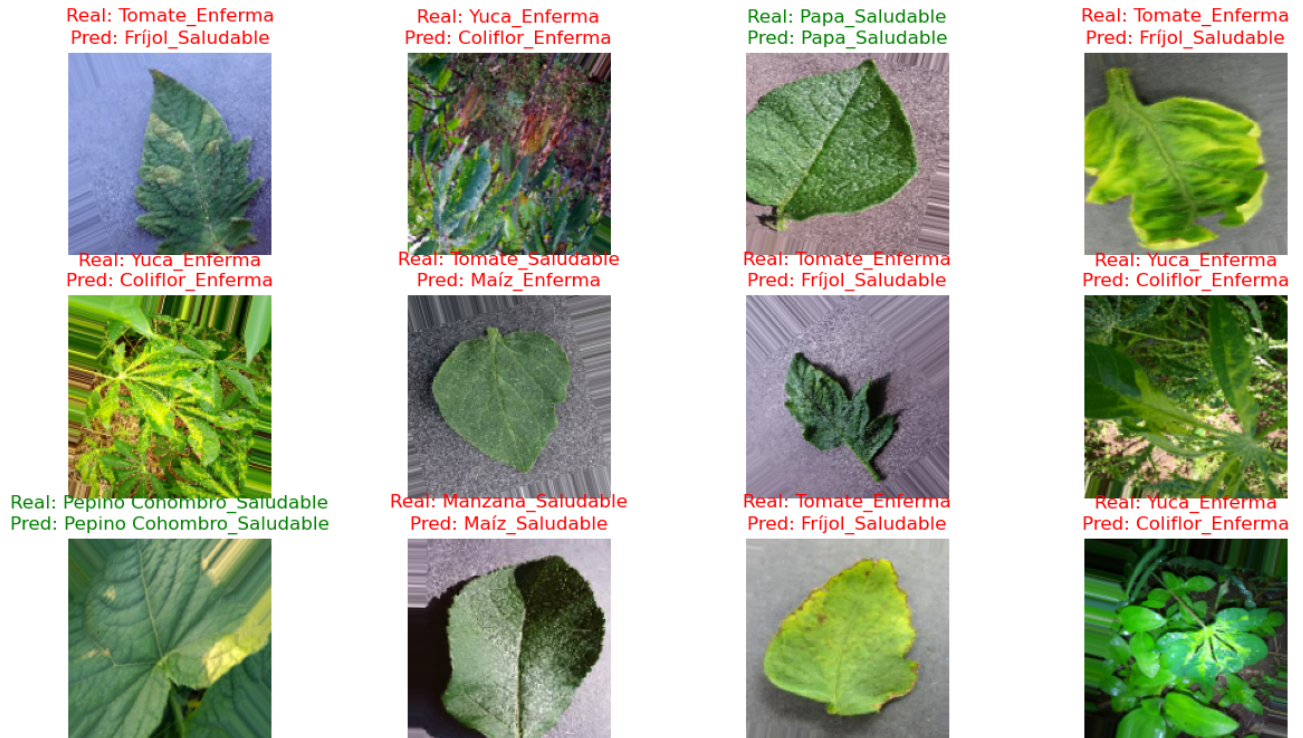
-----
ValueError                                Traceback (most recent call last)
Cell In[72], line 28
    26 plt.figure(figsize=(15, 8))
    27 for i in range(num_samples):
--> 28     plt.subplot(3, 4, i + 1)
    29     img_to_show = normalize_for_display(x_batch[i])
    30     plt.imshow(img_to_show)

File /opt/anaconda3/lib/python3.12/site-packages/matplotlib/pyplot.py:1534,
in subplot(*args, **kwargs)
    1531 fig = gcf()
    1533 # First, search for an existing subplot with a matching spec.
-> 1534 key = SubplotSpec._from_subplot_args(fig, args)
    1536 for ax in fig.axes:
    1537     # If we found an Axes at the position, we can reuse it if the us
er passed no
    1538     # kwargs or if the Axes class and kwargs are identical.
    1539     if (ax.get_subplotspec() == key
    1540         and (kwargs == {}
    1541             or (ax._projection_init
    1542                 == fig._process_projection_requirements(**kwargs)))
    ):

File /opt/anaconda3/lib/python3.12/site-packages/matplotlib/gridspec.py:589,
in SubplotSpec._from_subplot_args(figure, args)
    587 else:
    588     if not isinstance(num, Integral) or num < 1 or num > rows*cols:
--> 589         raise ValueError(
    590             f"num must be an integer with 1 <= num <= {rows*cols}, "
    591             f"not {num!r}"
    592         )
    593     i = j = num
    594 return gs[i-1:j]

ValueError: num must be an integer with 1 <= num <= 12, not 13

```

```
In [80]: # 3. Convertir etiquetas reales a one-hot
n_classes = y_pred_probs.shape[1]
y_true_onehot = to_categorical(y_true, num_classes=n_classes)

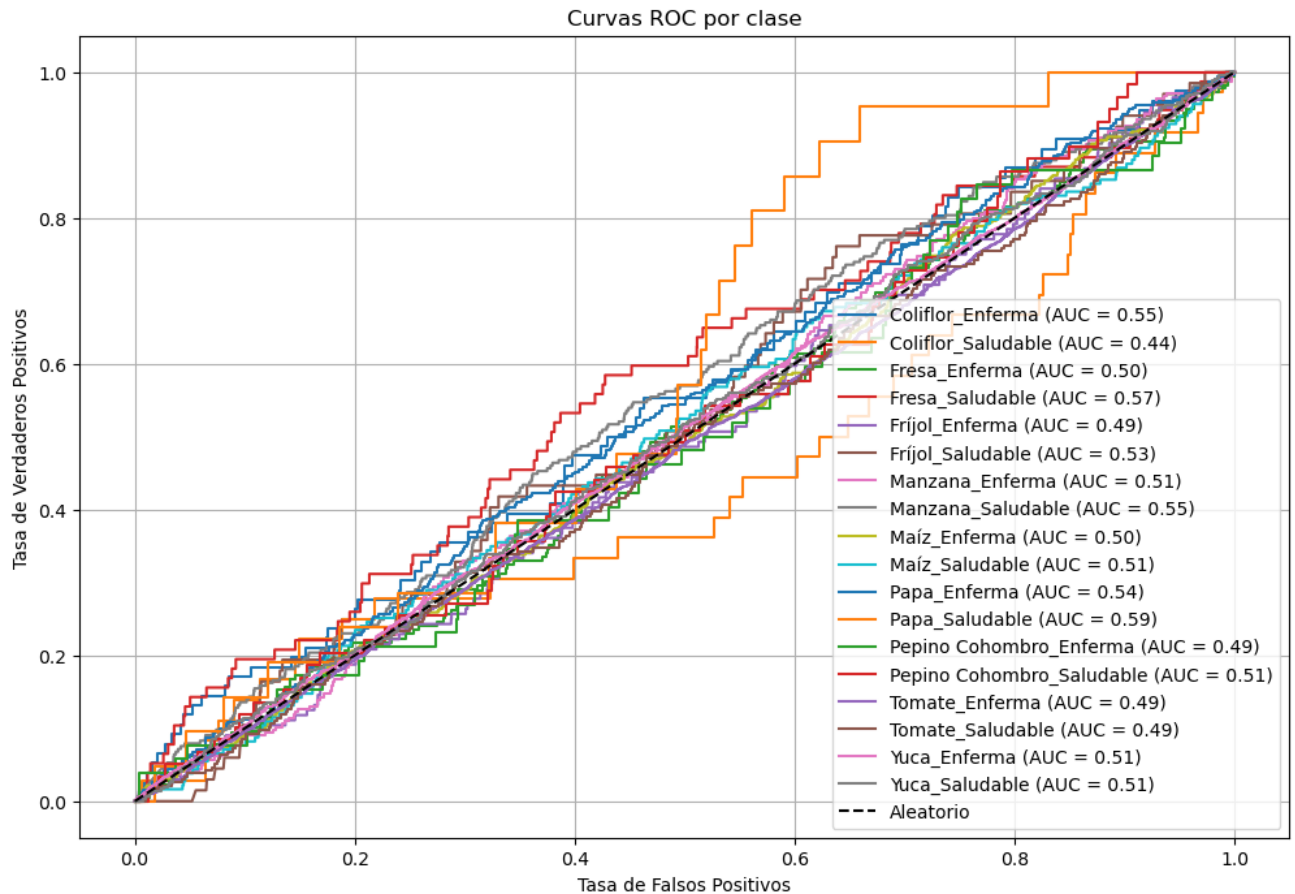
# 4. Calcular ROC y AUC por clase
fpr = dict()
tpr = dict()
roc_auc = dict()

for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_true_onehot[:, i], y_pred_probs[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# 5. Obtener nombres de clase
class_names = list(test_generator.class_indices.keys())

# 6. Graficar curvas ROC
plt.figure(figsize=(12, 8))
for i in range(n_classes):
    plt.plot(fpr[i], tpr[i], label=f'{class_names[i]} (AUC = {roc_auc[i]:.2f})')

plt.plot([0, 1], [0, 1], 'k--', label='Aleatorio')
plt.xlabel('Tasa de Falsos Positivos')
plt.ylabel('Tasa de Verdaderos Positivos')
plt.title('Curvas ROC por clase')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()
```



```
In [82]: from sklearn.metrics import roc_auc_score

# AUC macro (promedio de clases, todas igual de importantes)
auc_macro = roc_auc_score(y_true_onehot, y_pred_probs, average='macro')

# AUC micro (promedia sobre todas las instancias)
auc_micro = roc_auc_score(y_true_onehot, y_pred_probs, average='micro')

print(f"AUC Macro promedio: {auc_macro:.4f}")
print(f"AUC Micro promedio: {auc_micro:.4f}")
```

AUC Macro promedio: 0.5166

AUC Micro promedio: 0.5205

```
In [84]: # Asegúrate de tener esto:
# y_pred = np.argmax(y_pred_probs, axis=1)
# y_true = test_generator.classes

# Accuracy
accuracy = accuracy_score(y_true, y_pred)

# F1 Scores
f1_macro = f1_score(y_true, y_pred, average='macro')
f1_micro = f1_score(y_true, y_pred, average='micro')
```

```
f1_weighted = f1_score(y_true, y_pred, average='weighted')

print(f"Accuracy: {accuracy:.4f}")
print(f"F1 Score (macro): {f1_macro:.4f}")
print(f"F1 Score (micro): {f1_micro:.4f}")
print(f"F1 Score (weighted): {f1_weighted:.4f}")

# Detalle por clase
class_names = list(test_generator.class_indices.keys())
print("\nReporte de clasificación por clase:")
print(classification_report(y_true, y_pred, target_names=class_names))
```

```
Accuracy: 0.0221
F1 Score (macro): 0.0206
F1 Score (micro): 0.0221
F1 Score (weighted): 0.0222
```

Reporte de clasificación por clase:

	precision	recall	f1-score	support
Coliflor_Enferma	0.01	0.41	0.02	76
Coliflor_Saludable	0.01	0.03	0.01	36
Fresa_Enferma	0.01	0.01	0.01	179
Fresa_Saludable	0.03	0.10	0.04	77
Fríjol_Enferma	0.02	0.02	0.02	144
Fríjol_Saludable	0.01	0.36	0.02	67
Manzana_Enferma	0.04	0.04	0.04	245
Manzana_Saludable	0.05	0.08	0.06	265
Maíz_Enferma	0.05	0.03	0.04	417
Maíz_Saludable	0.01	0.02	0.01	183
Papa_Enferma	0.05	0.03	0.03	312
Papa_Saludable	0.00	0.00	0.00	21
Pepino Cohombro_Enferma	0.03	0.04	0.03	52
Pepino Cohombro_Saludable	0.00	0.00	0.00	59
Tomate_Enferma	0.23	0.01	0.02	2676
Tomate_Saludable	0.00	0.00	0.00	236
Yuca_Enferma	0.42	0.01	0.02	3008
Yuca_Saludable	0.00	0.00	0.00	419
accuracy			0.02	8472
macro avg	0.05	0.07	0.02	8472
weighted avg	0.23	0.02	0.02	8472

16. Guardar el modelo entrenado

```
In [87]: model.save('modelo_10_1_resnet50_PlantaEstado.h5')
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

```
In [89]: fin = datetime.datetime.now()
print(f"Fin: {fin.strftime('%Y-%m-%d %H:%M:%S.%f')}")

duracion = fin - inicio
print(f"Duración: {duracion}")
```

Fin: 2025-05-14 05:21:03.146248

Duración: 8:50:07.177904

```
In [91]: class_indices = train_generator.class_indices # o test_generator.class_indices
index_to_class = {v: k for k, v in class_indices.items()}
print(index_to_class)
with open("index_to_class_modelo10_1.json", "w") as f:
    json.dump(index_to_class, f, indent=4, ensure_ascii=False)
```

```
{0: 'Yuca_Enferma', 1: 'Yuca_Saludable', 2: 'Fresa_Saludable', 3: 'Papa_Enferma', 4: 'Maíz_Saludable', 5: 'Tomate_Enferma', 6: 'Manzana_Enferma', 7: 'Maíz_Enferma', 8: 'Tomate_Saludable', 9: 'Manzana_Saludable', 10: 'Fresa_Enferma', 11: 'Papa_Saludable', 12: 'Coliflor_Enferma', 13: 'Pepino Cohombro_Saludable', 14: 'Fríjol_Enferma', 15: 'Coliflor_Saludable', 16: 'Fríjol_Saludable', 17: 'Pepino Cohombro_Enferma'}
```

In []:

In []: